

Your project is an **Autonomous LLM Router that also acts like a payment-backed procurement agent**.

Instead of hardcoding “use Model X,” it treats every LLM call like a small commerce decision:

- **What am I trying to accomplish?** (math, quick Q&A, summarization, writing, reasoning, code)
- **What constraints do I have?** (max cost, latency, minimum quality, privacy/allowlist rules)
- **Which provider can satisfy this cheapest right now?** (Gemini vs Claude vs others)
- **Can I verify the output is good enough?** (deterministic checks / rubric checks)
- **Pay the chosen provider per request** using **USDC on Arc**, and keep an auditable receipt trail.

The core outcome: **users get the lowest-cost answer that still meets their quality requirements**, and every spend is controlled, enforceable, and traceable.

a

What it does end-to-end (in human terms)

1) You send a prompt + optional preferences

Example:

- Prompt: “2+2”
- Preference: “keep it cheap, max \$0.001”
- Or: “draft a sales email, premium quality allowed up to \$0.02”

2) The agent decides what kind of task this is

It runs a fast classifier to tag it:

- “trivial math” → low risk, cheap model OK
- “contract clause rewrite” → higher risk, higher quality needed
- “summarize 2 pages” → needs context length, maybe mid-tier model

This is the “agentic” part: **it interprets intent + risk + constraints**, not just the prompt string.

3) It checks your guardrails (bounded spending authority)

Before it does anything, it enforces:

- per-request cap

- daily cap
- allowed providers only
- emergency stop switch

So it can't accidentally burn money or pay unknown endpoints.

4) It “shops” among providers using quotes

Providers (really: your wrapper services around Gemini/Claude) expose something like:

- price per tokens
- estimated latency
- quality tier

The router compares them and chooses the cheapest provider that meets your needs.

5) It pays per request in USDC on Arc

This is the key hackathon requirement: **real stablecoin payment settlement**.

- The router sends USDC to the selected provider address on **Arc**
- It stores the **transaction hash**
- The provider verifies that payment happened (onchain) before it returns the completion

This creates a “pay-to-use” API interaction.

6) It executes the call and verifies output quality

After getting the model output, the router checks:

- deterministic verification for easy tasks (math evaluator, JSON schema validation)
- rubric checks for open-ended tasks (basic quality scoring)

If it fails verification and policy allows:

- it **escalates** to a higher-quality provider and pays again, but only if still within budget

7) It returns the answer + receipts + explanation

The UI shows:

- classification
- providers compared
- why the winner was chosen
- how much it cost
- the onchain tx hash (proof)

- whether escalation happened
-

What makes it a good product

Cost control that people actually feel

Most teams waste money by defaulting to “best model for everything.”
You give them **automatic savings** without manual tuning:

- cheap model for easy tasks
- premium only when needed

Policy-first (safe for real usage)

Teams can enforce:

- strict budgets
- allowlists
- task-based restrictions (“never use premium for low-risk tasks”)
- emergency stop

This matters when you let automation spend money.

Auditability and receipts

Every spend becomes:

- transparent
- verifiable
- attributable to a request_id

That’s valuable for finance/compliance, not just engineering.

Differentiation vs a normal “router”

A typical router:

- picks model based on heuristics
- charges you offchain
- no strong enforcement or receipts

Your system is different because it:

1. **treats inference like procurement** (choose provider based on live constraints + quotes)
 2. **settles per call in stablecoin** (machine-to-machine payments)
 3. **has verification + escalation** (quality control loop)
 4. **has enforceable budget guardrails** (bounded authority)
-

What you're really demonstrating for the hackathon

A complete "agentic commerce" loop:

- **autonomy:** agent chooses provider
- **coordination:** compares multiple provider offers
- **payments:** pays in USDC on Arc automatically
- **verification:** onchain receipt + quality checks
- **guardrails:** bounded spending authority

That's exactly the kind of story judges can validate instantly with a demo:

prompt → **routing decision** → **USDC transaction** → **result** → **receipt link**.